



SDU International Summer School

Introduction to R

Day 2 - Data structures

Paola Vazquez Castillo

04 August 2026

Contents

1	Introduction	2
2	Vector	2
3	Data frame	2
3.1	Selecting variables and observations	3
3.2	Expanding data frames	3
3.3	Missing values	4
4	Matrix	4
4.1	Selecting variables and observations	5
4.2	Expanding matrices	6
5	List	7
5.1	Selecting elements	8
6	Summary of data structures	9
7	Variable Arithmetic	9
7.1	Combining two variables	10
8	Importing a dataset	11
8.1	Where is R looking?	11
8.2	Choosing the right reading function	11
8.3	Reading a csv file	11
8.4	When it does not work	12
9	Getting to know your dataset	12
9.1	What structure is it, and how big?	12
9.2	What is inside it?	13

9.3	What type is a particular variable?	13
9.4	What values does it actually take?	13
10	Storing data in R	15
11	Glossary of functions	15
12	Exercise	16

1 Introduction

Data structures are specific ways of organizing data. Arranging data into structures makes them easier to manipulate, organize, process, and use. There are different types of structures, each serving a specific purpose. In this class we will discuss some of the most commonly used ones. Although in most of the cases you will work with already built sets (e.g. downloaded from a data source), it is important you can recognize what you are looking at.

2 Vector

As seen in Day 1, a vector is an ordered collection of data of the same type. Vectors are one-dimensional data structures.

```
#Numeric vector
x<-c(1:5)

#Character vector
y<-c("A", "B", "C", "D", "E")
```

3 Data frame

Data frames are the most popular data structure. They are two-dimensional objects, allowing to visualize data in a tabular form. Each column of the data frame represents a specific variable, consisting of vectors of the same length. Each row consists in a different observation.

Characteristics of a data frame:

- Each column and row should have a unique name.
- Each column has the same number of items.
- Each item of the same column must be the same data type.
- Different columns can have different data type.

We can create a data frame with the function `data.frame()`.

```
Beatles<-data.frame(Name = c("Paul", "John", "Ringo", "George"),
                    Instrument = c("Bass", "Guitar", "Drum", "Guitar"),
                    AgeToday = c(84,85,86,83))
```

Beatles

```
##      Name Instrument AgeToday
## 1   Paul      Bass      84
## 2   John     Guitar      85
```

```
## 3 Ringo      Drum      86
## 4 George    Guitar     83
```

In the above example, the data frame contains 3 variables (columns) and 4 observations (rows).

3.1 Selecting variables and observations

Specific variables can be selected either by using `$` or the square bracket option. The square brackets can also be used to select specific observation. As data frames are two-dimensional (rows and columns), if the square brackets are used, it must be specified if we want to select a row or a column such as `[row,column]`.

For example, if you want to select the first row, the position is defined as `[1,]`. If you want to select the first column, the position is defined as `[,1]`.

#Using \$ to select one variable

```
Beatles$Name
```

```
## [1] "Paul" "John" "Ringo" "George"
```

#Using [] to select variable from its name

```
Beatles[, "Name"]
```

```
## [1] "Paul" "John" "Ringo" "George"
```

#Using [] to select variable from its position

```
Beatles[,1]
```

```
## [1] "Paul" "John" "Ringo" "George"
```

#Using [] to select an observation from its position

```
Beatles[1,]
```

```
##   Name Instrument AgeToday
```

```
## 1 Paul      Bass      84
```

#Using [] and \$ to select an observation based on a specific variable

```
Beatles[Beatles$Name=="John",]
```

```
##   Name Instrument AgeToday
```

```
## 2 John      Guitar     85
```

```
Beatles[Beatles$Instrument=="Guitar",]
```

```
##   Name Instrument AgeToday
```

```
## 2 John      Guitar     85
```

```
## 4 George    Guitar     83
```

3.2 Expanding data frames

A new variable can be added to a data frame with `$`. The new variable/vector must contain the same number of observation as the other variables in the data frame.

Create a variable named status, indicating if the person is dead (1) or alive (0)

```
Beatles$Status<-c(0,1,0,1)
```

Beatles

```
##      Name Instrument AgeToday Status
## 1   Paul      Bass      84      0
## 2   John     Guitar     85      1
## 3  Ringo     Drum      86      0
## 4 George    Guitar     83      1
```

3.3 Missing values

If values are missing, the value should be replaced by: `NA`. When importing data, R will automatically replace with `NA` values it does not understand or missing values.

```
# Create a variable named about age at death
Beatles$Age_death<-c(NA, 40, NA, 58)
```

Beatles

```
##      Name Instrument AgeToday Status Age_death
## 1   Paul      Bass      84      0      NA
## 2   John     Guitar     85      1      40
## 3  Ringo     Drum      86      0      NA
## 4 George    Guitar     83      1      58
```

`NA` is not interpreted as a zero and it is also not an empty string (this confusion causes silent errors later).

```
# Sum a vector with NAs
sum(Beatles$Age_death)
```

```
## [1] NA
```

We will learn more about how to find and handle missing values in Day 4.

4 Matrix

A matrix is a rectangular arrangement of numbers in rows and columns. Matrices are two-dimensional, and unlike `data.frame()`, every cell must be of the same type (numeric, logical,...). That is what makes them useful: a matrix holds values of a single type, arranged in rows and columns.

A matrix expresses a single quantity across two variables. For example, we can express the number of people in a country by age (rows) and year (columns). Each cell is then the number of people of that age in that year. The rows and the columns are asking different questions of the same number.

You will rarely build a matrix by hand. You will meet them as output: a table of counts, a correlation matrix, the internals of a model,... What matters is recognizing one and knowing how to pull things out of it.

To create a matrix, we use the function `matrix()` with the arguments:

- The data or values
- The number of rows (`nrow`)

- The number of columns (ncol)
- `byrow`, whether the values are filled in row by row (TRUE) or column by column (FALSE, the default).

```
# Population of Denmark in thousands, by age group and year (approximate)
pop <- matrix(data= c(1290, 1330, 1300,
                     3260, 3320, 3400,
                     790, 885, 1120),
              nrow = 3, ncol = 3, byrow = TRUE)

pop
```

```
##      [,1] [,2] [,3]
## [1,] 1290 1330 1300
## [2,] 3260 3320 3400
## [3,] 790  885 1120
```

NB: `byrow = TRUE` lets you type the values the way they look on screen. Without it, R fills the matrix column by column, and you get a table that looks right but holds the wrong numbers in the wrong places. This is a very common and very silent mistake.

The rows and columns can be named with `rownames()` and `colnames()`.

```
rownames(pop) <- c("0-19", "20-64", "65+")
colnames(pop) <- c("2000", "2010", "2020")

pop
```

```
##      2000 2010 2020
## 0-19  1290 1330 1300
## 20-64 3260 3320 3400
## 65+   790  885 1120
```

An important aspect of a matrix is its dimension. The dimension can be found with the function `dim()`:

```
dim(pop)
```

```
## [1] 3 3
```

The output reads as rows first, columns second: three rows and three columns.

4.1 Selecting variables and observations

We use square brackets, with the row before the comma and the column after it. Leaving a position empty means “all of them”.

We can select by name of the columns and rows if we named them:

```
# How many people were aged 65+ in 2020?
pop["65+", "2020"]
```

```
## [1] 1120
```

```
# The whole age group across the three years (a row)
pop["65+", ]
```

```
## 2000 2010 2020
## 790 885 1120
```

```
# The whole age distribution in 2020 (a column)
pop[, "2020"]
```

```
## 0-19 20-64 65+
## 1300 3400 1120
```

We can also select by the position:

```
# How many people were aged 0-19 in 2000?
pop[1, 1]
```

```
## [1] 1290
```

```
# The whole age group 20-64 across the three years (a row)
pop[2, ]
```

```
## 2000 2010 2020
## 3260 3320 3400
```

```
# The whole age distribution in 2020 (a column)
pop[,3]
```

```
## 0-19 20-64 65+
## 1300 3400 1120
```

Because every cell holds the same quantity (i.e. population), we can total in either direction. `colSums()` adds down each column, `rowSums()` across each row.

```
# Total population in each year
colSums(pop)
```

```
## 2000 2010 2020
## 5340 5535 5820
```

The columns sum to something meaningful: the total population of the country in that year. Now look at the rows:

```
rowSums(pop)
```

```
## 0-19 20-64 65+
## 3920 9980 2795
```

R gives an answer, but the answer is not a population. It adds the same people several times, once in each year they appear.

NB: R will compute anything you ask it for, whether or not the result means something. Deciding whether a number is meaningful is your job.

4.2 Expanding matrices

A row or a column can be added to a matrix by using the functions `rbind()` to add a row and `cbind()` to add a column. The vector being added to the original matrix must respect its dimensions.

```

# Add a column: a projection for a new year.
# The new vector needs as many values as the matrix has rows.
pop_projected <- cbind(pop, "2030" = c(1180, 3390, 1420))
pop_projected

##          2000 2010 2020 2030
## 0-19    1290 1330 1300 1180
## 20-64   3260 3320 3400 3390
## 65+      790  885 1120 1420

# Add a row: the total population in each year.
# The new vector needs as many values as the matrix has columns.
pop_total <- rbind(pop, "Total" = colSums(pop))
pop_total

##          2000 2010 2020
## 0-19    1290 1330 1300
## 20-64   3260 3320 3400
## 65+      790  885 1120
## Total   5340 5535 5820

```

NB: this second one is worth pausing on. It is convenient, and it is a habit carried over from spreadsheets, but the matrix now holds two different kinds of row: three age groups and one summary. Any calculation applied to the whole matrix from now on will treat that total as if it were another age group, and will silently double the population. Keep your data and your summaries in separate objects.

5 List

Lists are objects which can contain multiple elements of different types and different structures: a number, a character, a matrix, a data frame, and so on. A list is created using `list()` function.

```

mylist<-list(Beatles,pop, "Notes about the data")

mylist

## [[1]]
##      Name Instrument AgeToday Status Age_death
## 1   Paul      Bass      84      0      NA
## 2   John     Guitar     85      1      40
## 3  Ringo      Drum      86      0      NA
## 4 George    Guitar     83      1      58
##
## [[2]]
##          2000 2010 2020
## 0-19    1290 1330 1300
## 20-64   3260 3320 3400
## 65+      790  885 1120
##
## [[3]]
## [1] "Notes about the data"

```

We can name the elements of the list with the function `names()`

```
names(mylist)<-c("Dataframe", "Matrix", "Note")
```

```
mylist
```

```
## $Dataframe
##      Name Instrument AgeToday Status Age_death
## 1   Paul      Bass      84      0      NA
## 2   John     Guitar     85      1      40
## 3  Ringo      Drum     86      0      NA
## 4 George     Guitar     83      1      58
##
## $Matrix
##      2000 2010 2020
## 0-19 1290 1330 1300
## 20-64 3260 3320 3400
## 65+   790  885 1120
##
## $Note
## [1] "Notes about the data"
```

This is the structure you are least likely to build yourself and most likely to receive. When you fit a statistical model on Day 8, what R gives back is a list: the coefficients, the residuals, the data used, the formula, all bundled into one object. Knowing that a list is a container of unrelated pieces, and knowing how to reach into it, is what lets you pull out the part you need.

5.1 Selecting elements

To select a specific object within the list, we use double square brackets `[[ ]]` or `$`.

```
# Select from position in the list
```

```
mylist[[1]]
```

```
##      Name Instrument AgeToday Status Age_death
## 1   Paul      Bass      84      0      NA
## 2   John     Guitar     85      1      40
## 3  Ringo      Drum     86      0      NA
## 4 George     Guitar     83      1      58
```

```
#Select from the name
```

```
mylist$Matrix
```

```
##      2000 2010 2020
## 0-19 1290 1330 1300
## 20-64 3260 3320 3400
## 65+   790  885 1120
```

NB: single brackets `[ ]` also work on a list, but they give you back a shorter list rather than the object inside it. `mylist[1]` is a list containing the data frame, `mylist[[1]]` is the data frame itself. If a function complains that it was given a list when it expected data, this is usually why.

6 Summary of data structures

Structure	Contains	Same type?	Where you find it
Vector	A sequence of values	Yes	A single variable, a column of a dataset
Data frame	Columns of different types	No (across columns)	Almost all real data
Matrix	Values in rows and columns	Yes	Correlation matrices, tables of counts
List	Objects of any type or structure	No	Model output, anything with mixed pieces

Real data will nearly always reach you as a data frame. The other data structures matter because functions return them, and because you need to recognize what you are looking at when they do.

7 Variable Arithmetic

On day 1, we saw basic arithmetic applied to single element data. The same operations work on a whole variable at once.

```
# Using our Beatles data frame
Beatles$AgeToday
```

```
## [1] 84 85 86 83
```

```
# Add 5 years to everyone's age
Beatles$AgeToday + 5
```

```
## [1] 89 90 91 88
```

Notice what happened: one instruction, four results. R applied the operation to every element of the variable without being asked to repeat itself. This is called *vectorisation*, and it is one of the main reasons R is convenient for data work. In many programming languages you would need a loop to do this. On Day 3 you will learn to write loops, and you will also learn how often you can avoid them.

Most of the time we do not want to just see the result, we want to keep it. Assigning it to a new name with `$` creates a new variable in the data frame.

```
# Calculate age in decades
Beatles$Age_in_decades <- Beatles$AgeToday / 10
Beatles
```

```
##      Name Instrument AgeToday Status Age_death Age_in_decades
## 1   Paul      Bass      84      0      NA      8.4
## 2   John    Guitar      85      1      40      8.5
## 3  Ringo     Drum      86      0      NA      8.6
## 4 George    Guitar      83      1      58      8.3
```

```
# Calculate age in months
Beatles_Age_in_months <- Beatles$AgeToday * 12
Beatles
```

```
##      Name Instrument AgeToday Status Age_death Age_in_decades
## 1   Paul      Bass      84      0      NA      8.4
## 2   John     Guitar     85      1      40      8.5
## 3  Ringo      Drum      86      0      NA      8.6
## 4 George     Guitar     83      1      58      8.3
```

The age in months was not saved as part of the Beatles dataframe but as an object on its own, that is why it does not appear when looking at the dataframe Beatles.

7.1 Combining two variables

The most useful case is a new variable calculated from two existing ones. R works element by element: the first value with the first value, the second with the second, and so on.

```
# How many years ago did each of them die?
Beatles$Years_since_death <- Beatles$AgeToday - Beatles$Age_death
Beatles$Years_since_death
```

```
## [1] NA 45 NA 25
```

The calculation gave a result for John and George, and NA for Paul and Ringo. This is correct: they are alive, so `Age_death` is missing, and a subtraction involving an unknown value is unknown. R does not guess and it does not skip. Any arithmetic touching an NA returns NA, which is why finding your missing values before you calculate matters, and we come back to it on Day 4.

The new variable also carries no memory of how it was made. It is now just four numbers sitting in the data frame. If your calculation was wrong, nothing in the data will tell you. Always look at the result (`head()`, or the data frame itself) and ask whether the numbers are plausible before you use them.

```
head(Beatles)
```

```
##      Name Instrument AgeToday Status Age_death Age_in_decades Years_since_death
## 1   Paul      Bass      84      0      NA      8.4              NA
## 2   John     Guitar     85      1      40      8.5              45
## 3  Ringo      Drum      86      0      NA      8.6              NA
## 4 George     Guitar     83      1      58      8.3              25
```

NB: this pattern, `data$new_variable <- data$variable_a - data$variable_b`, is one you will use constantly. It is worth being able to write it without thinking.

Arithmetic also works on a matrix, applying the operation to every cell.

```
# Express the figures in millions instead of thousands
round(pop / 1000, 2)
```

```
##      2000 2010 2020
## 0-19  1.29 1.33 1.30
## 20-64 3.26 3.32 3.40
## 65+   0.79 0.88 1.12
```

8 Importing a dataset

On Day 1 we opened a file in the quickest way possible. Here we do it properly, because this is where most of the errors appear when one is dealing with real data.

8.1 Where is R looking?

R does not search all your computer for a file. It looks in one folder, the working directory, and if the file is not there it returns an error. To check the folder where R currently is looking at:

```
getwd()
```

To change it, use `setwd()` with the path to your folder, or the menu Session > Set Working Directory > Choose Directory.

```
# setwd("C:/Users/.../summer_school")
```

Two points about paths. R uses forward slashes / even on Windows, so a path copied from the Windows explorer with backslashes will not work and has to be turned around. And the path goes in quotation marks.

NB: put `setwd()` at the top of your script, once. Then every file you read or write in that session is relative to that folder, and you never type a full path again.

8.2 Choosing the right reading function

The function you need depends on the file format, which you read off the extension at the end of the file name.

Extension	What it is	Function
.csv	Comma separated values	<code>read.csv()</code>
.csv	Semicolon separated (common in Europe)	<code>read.csv2()</code>
.txt	Plain text	<code>read.table()</code>
.RData	R workspace, possibly several objects	<code>load()</code>
.rds	A single R object	<code>readRDS()</code>
.xlsx	Excel	<code>read_excel()</code> , needs a package
.sav, .dta	SPSS, Stata	<code>read_sav()</code> , <code>read_dta()</code> , need a package

The last two rows need packages, which we learn about on Day 4.

8.3 Reading a csv file

As an example we will read the dataset called `WC2026.csv`, this file contains information for the 48 countries that participated in this year's Football World Cup. The source is: <https://footystats.org/international/world-cup/datasets>

```
data <- read.csv("WC2026.csv", header = TRUE, sep = ",")
head(data)
```

```
##      country matches_played wins losses goals_scored goals_conceded
## 1 Switzerland           6    3     1           10           6
## 2   Austria            4    1     2            6           9
## 3    Spain             8    7     0           14           1
## 4  Portugal            5    2     1            8           3
## 5   Norway            6    4     2           13          11
## 6   Germany            4    2     1           11           5
##  cards_total confederation
## 1           8          UEFA
## 2           5          UEFA
## 3           6          UEFA
## 4           7          UEFA
## 5           3          UEFA
## 6           3          UEFA
```

The arguments worth knowing:

- **header**: is the first row the variable names? Usually `TRUE`.
- **sep**: what separates the columns? A comma by default.
- **skip**: how many rows to ignore at the top, useful when a file starts with a title or a note before the actual table.

8.4 When it does not work

The most common error you will see is:

```
Error in file(file, "rt") : cannot open the connection
In addition: Warning message:
cannot open file 'WC2026.csv': No such file or directory
```

This almost always means that R looked in its working directory and did not find that file. Three checks, in order:

1. Where am I? `getwd()`
2. What is actually in this folder? `list.files()`
3. Is the name exactly right? Remember R is case sensitive, so `WC2026.csv` and `wc2026.csv` are different files.

9 Getting to know your dataset

Importing the data is not the same as knowing what you imported. Before doing anything else, find out what R thinks it just read. Four questions, four sets of functions.

9.1 What structure is it, and how big?

```
class(data)
```

```
## [1] "data.frame"
```

```
dim(data)
```

```
## [1] 48  8
```

`dim()` reads rows first, columns second. If you want them separately, or the variable names:

```
nrow(data)
```

```
## [1] 48
```

```
ncol(data)
```

```
## [1] 8
```

```
names(data)
```

```
## [1] "country"      "matches_played" "wins"           "losses"
## [5] "goals_scored" "goals_conceded" "cards_total"    "confederation"
```

9.2 What is inside it?

`str()` (for structure) is the single most useful of these. It gives you the structure, the dimensions, the type of every variable and the first few values, all at once.

```
str(data)
```

```
## 'data.frame':   48 obs. of  8 variables:
## $ country      : chr  "Switzerland" "Austria" "Spain" "Portugal" ...
## $ matches_played: int   6 4 8 5 6 4 8 6 4 8 ...
## $ wins         : int   3 1 7 2 4 2 6 3 2 6 ...
## $ losses       : int   1 2 0 1 2 1 1 1 0 2 ...
## $ goals_scored  : int  10 6 14 8 13 11 20 14 11 20 ...
## $ goals_conceded: int   6 9 1 3 11 5 12 7 5 10 ...
## $ cards_total   : int   8 5 6 7 3 3 10 8 3 6 ...
## $ confederation : chr   "UEFA" "UEFA" "UEFA" "UEFA" ...
```

Read it carefully the first time. `chr` is character, `int` is integer, `num` is numeric, `Factor` is a factor. This is where you find out that a variable you expected to be a number was read as text, which happens more often than you would like and breaks every calculation afterwards.

9.3 What type is a particular variable?

```
class(data$country)
```

```
## [1] "character"
```

```
class(data$matches_played)
```

```
## [1] "integer"
```

9.4 What values does it actually take?

For a categorical variable, `unique()` lists every distinct value, with no repetitions.

```
unique(data$confederation)
```

```
## [1] "UEFA"      "AFC"      "CAF"      "CONMEBOL" "CONCACAF" "OFC"
```

This tells you three useful things at once: how the categories are spelled, how many there are, and whether the same category has been entered in more than one way (“uefa” and “UEFA” are

two different values to R).

Since `confederation` is a categorical variable, we can convert it into a factor, as we saw on Day 1.

```
data$confederation <- as.factor(data$confederation)
class(data$confederation)
```

```
## [1] "factor"
```

```
levels(data$confederation)
```

```
## [1] "AFC"      "CAF"      "CONCACAF" "CONMEBOL" "OFC"      "UEFA"
```

For a numeric variable, listing every distinct value is useless: you would get hundreds of numbers. What you want is the shape of the distribution, which is what `summary()` gives.

```
summary(data$wins)
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## 0.000  0.000   1.000   1.667   2.250   7.000
```

Summary shows the minimum, maximum, mean, median and quartiles, plus a count of missing values if there are any.

NB: choosing between `unique()` and `summary()` is a question about the variable, not about the function. `unique()` answers “which categories exist here”, so it suits character and factor variables with a limited set of values. `summary()` answers “how are these numbers spread out”, so it suits numeric variables. Running `summary()` on a character variable gives you almost nothing, and running `unique()` on a continuous variable gives you a wall of numbers. Look at the type first, then choose.

`summary()` also works on the whole data frame at once, giving a column by column overview:

```
summary(data)
```

```
##      country  matches_played      wins      losses
## Length :48   Min. :3.000   Min. :0.000   Min. :0.000
## N.unique :48   1st Qu.:3.000   1st Qu.:0.000   1st Qu.:1.000
## N.blank  :0   Median :4.000   Median :1.000   Median :2.000
## Min.nchar:3   Mean :4.333   Mean :1.667   Mean :1.667
## Max.nchar:22  3rd Qu.:5.000   3rd Qu.:2.250   3rd Qu.:2.000
##              Max. :8.000   Max. :7.000   Max. :3.000
##
## goals_scored goals_conceded cards_total confederation
## Min. : 0.000   Min. : 1.000   Min. : 1.000   AFC      : 9
## 1st Qu.: 2.000   1st Qu.: 4.000   1st Qu.: 4.000   CAF      :10
## Median : 5.000   Median : 6.000   Median : 6.000   CONCACAF: 6
## Mean : 6.417   Mean : 6.417   Mean : 6.289   CONMEBOL: 6
## 3rd Qu.:10.000   3rd Qu.: 9.000   3rd Qu.: 8.000   OFC      : 1
## Max. :20.000   Max. :12.000   Max. :15.000   UEFA     :16
##              NAs :3
```

We come back to this on Day 4, together with how to find missing values systematically.

10 Storing data in R

As you work with the data, modify the data frames or matrices, and produce results, you will eventually want to save your data. This will allow you to work later on with it or share it with other people.

The easiest way to save the data is to save it into an RData file with the function `save()`. The necessary arguments are:

- The name of the object to be saved.
- The name of the RData file.

```
#setwd()
save(mylist, file="Example.RData")
```

To open back the file, you need to use the function `load()`.

```
#setwd()
load("Example.RData")
```

Be sure you set up your work directory to tell R where to save and load the data. RData files can, however, only be read by R. There are other options to save data, including `write.csv()` and `write.table()`, saving the data into a csv file and txt file, respectively. However, the last two functions can only save data frames and matrices. Sometimes it is convenient to store data as a RData file, e.g. many rows and columns, because this allows loading the data and working with it to be faster.

```
#setwd()
write.csv(Beatles, file="Beatles.csv", row.names = FALSE)
```

11 Glossary of functions

The functions introduced today. Everything from Day 1 still applies. If you cannot say what one of these does, that is what to review.

Function	What it does
Building structures	
<code>data.frame()</code>	Creates a data frame
<code>matrix()</code>	Creates a matrix
<code>list()</code>	Creates a list
Naming	
<code>names()</code>	Names the columns of a data frame, or the elements of a list
<code>rownames()</code> , <code>colnames()</code>	Names the rows and columns of a matrix
Checking what you have	
<code>class()</code>	Type of an object or of a variable
<code>str()</code>	Structure, size and variable types, all at once
<code>dim()</code>	Rows and columns
<code>nrow()</code> , <code>ncol()</code>	Number of rows, number of columns
Selecting	
<code>[row, column]</code>	Selects from a data frame or a matrix
<code>\$</code>	Selects a variable, or an element of a list

Function	What it does
<code>[[]]</code>	Selects an element of a list
Combining and totalling	
<code>rbind()</code> , <code>cbind()</code>	Adds a row, adds a column
<code>colSums()</code> , <code>rowSums()</code>	Totals down each column, across each row
Exploring the values	
<code>unique()</code>	The distinct values of a categorical variable
<code>summary()</code>	The distribution of a numeric variable, or of a whole dataset
Finding your files	
<code>getwd()</code>	Which folder R is working in
<code>setwd()</code>	Changes that folder
<code>list.files()</code>	What is actually in the folder
Reading and saving data	
<code>read.csv()</code> , <code>read.csv2()</code>	Opens a csv (comma separated, semicolon separated)
<code>read.table()</code>	Opens a txt file
<code>load()</code>	Opens an .RData file
<code>save()</code>	Saves objects into an .RData file
<code>write.csv()</code>	Saves a data frame as a csv

12 Exercise

- Create a data frame with the variables and values in the table below.
 - Select the Age variable.
 - What is the average age?
 - Select the salary variable.
 - What is the maximum salary?
 - Add a variable about the number of children

Name	Age	Sex	Country	Salary
Peter	51	Male	USA	4400
Sara	32	Female	Italy	2900
Adam	29	Male	Denmark	3200
Lisa	25	Female	Portugal	1800
Joseph	43	Male	France	3850

Name	Children
Peter	3
Sara	1
Adam	0
Lisa	NA
Joseph	4

- Create 3 vectors, each one containing 4 elements.
 - Combine the 3 vectors to obtain a matrix with 4 rows and 3 columns.

- Change the row names to “A”, “B”, “C”, and “D” and the column names to “X”, “Y”, “Z”.
- Select the column X.
- Select the row C.
- What is the element in row B column Z?
- From the World Cup database:
 - Set working directory and import `WC2026.csv`
 - How many observations and variables does it have? What type is each variable?
 - Which confederations are represented in the data?
 - Show the last 10 observations of the dataframe
 - Create a variable that shows the number of draws (hint: the data set includes how many games were played, how many won and how many lost).
 - Create a variable that shows what percentage of the played games were lost.
 - Summarize your two new variables. What do they show?